Data Engineer & Evaluation Specialist

NASA C-MAPSS Turbofan Remaining Useful Life (RUL) Production-grade data pipeline, leakage control, and shared evaluation protocol

Deliverable scope

This document operationalizes the responsibilities of the Data Engineer and Evaluation Specialist for the project plan. It defines the data contract, fold-safe preprocessing, target construction, feature/window preparation, automated leakage tests, metric implementation, and the reporting artifacts that every modeling stream must consume. It does not claim experimental performance: no model has been trained or ranked.

Non-negotiable principle: every fitted transformation and every model-selection decision must be learned only from the training engines of the current fold. The official test set remains untouched until the final, coordinated run.

Workstream	Concrete outcome	Acceptance evidence
Dataset ingestion	Canonical schema and validation of FD001–FD004 files	Schema checks, row/order checks, engine-cycle integrity
RUL target	Separate rul_true, rul_train, rul_clip_eval	Unit tests and target lineage documentation
Split protocol	Engine-level GroupKFold manifest	folds.json shared by every model
Preprocessing	Train-only learned transforms; causal features	Fit/transform provenance and leakage tests
Evaluation	RMSE, MAE, NASA Score, Last-20 and timing	Manual-oracle tests and OOF prediction files

1. Dataset contract and ingestion

NASA C-MAPSS is handled as a set of independent engine trajectories, not as exchangeable rows. A record is an observation at engine i and cycle t, with three operating settings and 21 sensor channels. The pipeline must preserve the engine boundary and temporal order at every stage.

Subset	Train engines	Test engines	Operating conditions	Failure modes	Role
FD001	100	100	1	1	Primary development subset
FD002	260	259	6	1	Multi-condition validation
FD003	100	100	1	2	Second failure-mode validation
FD004	249	248	6	2	Hardest distributional setting

Canonical columns

Column group	Canonical naming	Validation rule
Identifiers	dataset, split, engine_id, cycle	engine_id integer; cycle strictly increasing within engine; no duplicated (dataset, split, engine_id, cycle)
Operating settings	setting_1, setting_2, setting_3	numeric and preserved before condition clustering
Sensor channels	sensor_01 … sensor_21	numeric; missingness summarized before imputation
Targets	rul_true, rul_train, rul_clip_eval	never overwrite one target with another
Metadata	fold, seed, feature_version, pipeline_version	written to every prediction/result artifact

Ingestion sequence

• Read raw whitespace-delimited files with explicit 26-column names; reject malformed row counts or nonnumeric sensor/settings values.

• Sort by dataset, split, engine_id, cycle; assert cycle uniqueness and monotonicity per engine.

• Attach source metadata: raw-file checksum, data version, ingestion timestamp, and row/engine counts.

• Produce an immutable canonical parquet/CSV layer. All later transformations are derived artifacts, never edits to raw data.

2. RUL target engineering

Target construction is intentionally separated into training and evaluation targets. This prevents the common error of reporting optimistic metrics on an artificially clipped target.

Target	Definition	Permitted use	Forbidden use
rul_true	For training trajectories: max_cycle(engine) − cycle. For official test: supplied terminal RUL plus distance to observed terminal cycle where required by the evaluation convention.	All principal metrics and error analysis	Training objective when the protocol calls for capped RUL
rul_train	min(rul_true, 125)	Training label for every model under the shared piecewise-linear cap	Primary RMSE/MAE/NASA reporting
rul_clip_eval	min(rul_true, c_eval), explicitly documented	Appendix-only sensitivity reporting	Replacing rul_true in main comparison

Target equations

For each training engine i at cycle t: $rul_true_{i,t}=T_i-t$, where T_i is the last observed failure cycle for that engine. The shared training target is $rul_train_{i,t}=min(rul_true_{i,t},125)$. The cap is fixed across models; 110 and 150 are separate LightGBM ablations, not per-model tuning knobs.

Operational checks

- At each engine’s final training cycle, rul_true must equal 0.
- Within an engine, rul_true must decrease by exactly one per cycle.
- rul_train must be within [0, 125] and equal rul_true whenever rul_true ≤ 125.
- The test RUL reference file must have exactly one entry per test engine and be joined only by deterministic engine order/ID validation.

Presentation note: emphasize that clipping is a training-inductive bias, whereas rul_true is the operational quantity used to judge safety and accuracy.

3. Fold-safe preprocessing architecture

Every fold is a complete fitted pipeline. No scaler, imputer, condition clusterer, feature selector, or learned sensor filter may consume validation engines. The same principle applies inside hyperparameter optimization: inner validation decisions must be made only from the outer-training partition.

Stage	Fit scope	Transform scope	Artifact
1. Sensor screening	Outer-train engines only	Train/validation/test using retained sensor list	retained_sensors.json
2. Operating-condition model	Outer-train settings only for FD002/FD004	Assign train/validation/test with frozen clusterer	condition_model.joblib
3. Imputation/scaling	Outer-train rows only; preferably condition-aware	Apply frozen statistics	preprocessor.joblib
4. Feature engineering	No learned future information; histories per engine	Causal transform independently by engine	feature_schema.json
5. Windowing	No fitting; enforce engine-local ordering	All partitions independently	window_manifest.parquet

Sensor treatment

• Constant or near-constant sensors are identified with variance calculated only on the training portion of the fold. The retained feature list is then frozen.

• For FD002 and FD004, condition-aware normalization is required because changing operating regimes shift sensor distributions. Cluster operating settings on fold-train data only; fit a scaler per condition or include condition representation in a leakage-safe pipeline.

• Missing-value policy must be explicit. If raw C-MAPSS has no missing values, the pipeline still exposes an imputer for reproducibility and future deployment data; it is fitted only on fold-train data.

Causal feature set

Family	Windows / definition	Causality guard
Current-state	Scaled sensor/settings values at cycle t	Uses t only
Rolling statistics	mean and std over 5, 10, 30 cycles	rolling window is trailing: [t−w+1, t]
Trend	Slope estimated over trailing window	No centered window; no future points
Relative baseline	value(t) − mean(first 5 cycles)	Only if the same 5-cycle warm-up is available at inference
Sequence input	Trailing 30–50-cycle tensor	Pad short histories and supply a mask; never borrow rows from another engine

4. Engine-level validation design

The unit of generalization is an unseen engine. Consequently, row-wise random splitting is prohibited: it would place neighboring cycles of the same engine into both train and validation and substantially inflate performance.

Experiment	Question answered	Split protocol	Status
A: Within-subset	Can the method generalize to unseen engines under the same subset distribution?	5-fold GroupKFold on engine_id, independently for each FD subset	Mandatory
B: Cross-subset transfer	How much does performance degrade under different operating conditions or failure modes?	Train on one subset, evaluate on another; e.g., FD001 → FD003	Supplementary after A
Official test	Final held-out benchmark	One coordinated execution after model/protocol lock	Single final run only

Fold manifest specification

The Data Engineer creates a versioned folds.json before model development. It must map each dataset and fold to explicit arrays of train_engine_ids and valid_engine_ids, plus random_state, split method, creation timestamp, and manifest hash. All modelers load this artifact rather than generating their own splits.

Nested tuning protocol

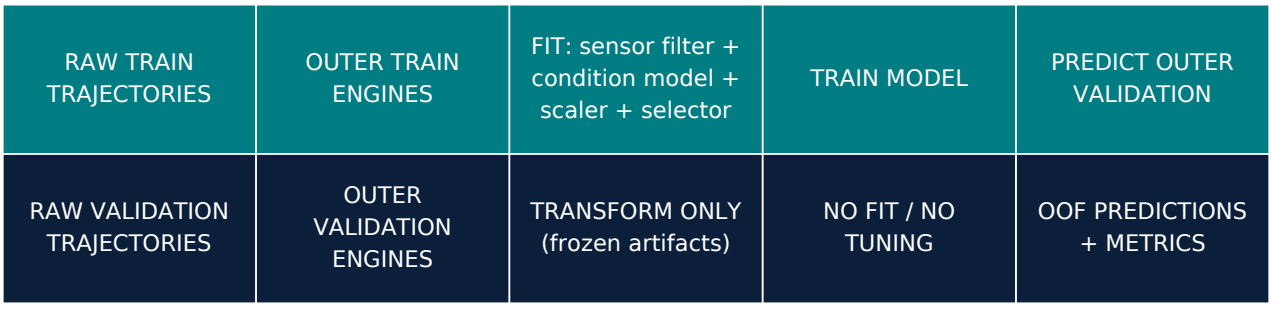
• Outer fold: reserved only for unbiased validation and out-of-fold (OOF) predictions.

• Inner folds: constructed only from the outer-train engines for Optuna/model selection.

• Final fold fit: refit preprocessing and model on all outer-train engines using the chosen inner configuration; predict outer-validation engines exactly once.

• Aggregate only OOF predictions for fold-comparable RMSE, MAE, NASA Score, and Last-20 metrics.

Chart: data-flow and leakage boundary



5. Shared evaluation framework

All metrics are computed from a single library module that consumes aligned arrays of rul_true and rul_pred. The error sign convention is fixed: $d=\hat{y}-y$. A positive error is an optimistic RUL estimate and receives the harsher NASA penalty.

Metric	Definition	Interpretation / reporting
MAE	$mean(\|d\|)$	Typical absolute cycle error; robust and easily interpretable
RMSE	$sqrt(mean(d^2))$	Penalizes large misses; principal global error metric
NASA Score	if d<0: $e^{-d/13}-1$; if d≥0: $e^{d/10}-1$	Asymmetric: optimistic predictions cost more; lower is better
Last-20 MAE/RMSE	Metric restricted to rul_true ≤ 20, using OOF predictions	End-of-life reliability, not “last 20 predictions per engine”
Coverage90 & width90	fraction of truths in [lo90, hi90]; mean(hi90−lo90)	Always report both calibration and sharpness
Cost	fit seconds, inference seconds, model parameters	Resource/operational trade-off; timed on fixed environment

NASA Score asymmetry chart

Error d (cycles)	-20	-10	-5	0	+5	+10	+20
Penalty behavior	underestimate	underestimate	underestimate	neutral	optimistic	optimistic	optimistic

At equal magnitude, positive errors grow faster because the denominator is 10 rather than 13. In the presentation, connect this to maintenance risk: predicting too much remaining life can defer intervention beyond a safe point.

Required output files

File	Granularity	Required columns
metrics_summary.csv	one row per model × dataset × fold × seed	model, dataset, fold, seed, rmse, mae, nasa_score, rmse_last20, mae_last20, coverage90, width90, train_seconds, infer_seconds
predictions_oof.csv	one row per scored observation	model, dataset, fold, engine_id, cycle, rul_true, rul_pred, lo90, hi90, split=OOF
run_manifest.json	one per run	git commit, data hash, folds hash, feature schema hash, environment, seed, timestamps
leakage_test_report.json	one per pipeline execution	test name, status, details, failing IDs if any

6. Quality gates, tests and execution order

The pipeline should fail closed: training cannot begin when a validation contract fails. Tests are not documentation; they are executable gates in CI or at minimum a mandatory pre-run command.

Test	Method	Pass condition	Risk prevented
Engine separation	Set intersection of outer train/valid engine IDs	Empty for every fold	Same-engine leakage
Window boundary	Check every row/window has a single engine_id and consecutive cycles	No cross-engine window; no nonconsecutive leakage	Trajectory mixing
Feature causality	Perturb future cycles after t and recompute features at t	Feature vector at t is unchanged	Future information leakage
Fit-scope audit	Log index/engine IDs passed to every fit call	All fit IDs subset of current train engines	Scaler/imputer/clusterer leakage
Target integrity	Check target equations and cap invariants	All engines satisfy monotonicity/terminal rules	Wrong labels
Metric oracle	Hand-calculated toy arrays for RMSE/MAE/NASA/coverage	Exact numeric match within tolerance	Sign/formula errors
Manifest consistency	Compare folds.json hash across runs	Exact hash match	Unfair comparison

Recommended project structure

Path	Purpose
data/raw/	Immutable original C-MAPSS files and RUL reference files
data/processed/	Canonical data, fold-specific artifacts, window manifests
src/data/	Loader, schema validation, RUL calculation, preprocessing, window builder
src/evaluation/	Metrics, aggregation, plotting, output writer
tests/	Leakage, target, metric-oracle, and contract tests
configs/	Dataset and experiment YAML files; fixed seed and versions
artifacts/	folds.json, feature schemas, manifests, fitted fold artifacts

Execution sequence for the specialist

- Freeze schema and build canonical data layer; publish engine and cycle inventory.
- Implement RUL targets and target tests; obtain team sign-off on definitions.
- Generate and freeze folds.json; distribute the manifest hash.
- Implement fold-scoped preprocessing and causal feature/window builders.
- Implement metrics and their oracle tests; produce one synthetic end-to-end example.
- Enable common artifact writers; verify all model streams can emit compatible OOF predictions.
- Only then support model training, error analysis, cross-subset evaluation, and the single official test run.

7. Presentation-ready narrative

Use the following framing in the defense. It demonstrates that the project is designed as an experimental system rather than a collection of disconnected models.

Slide / section	Core message	Evidence to show
Problem framing	RUL is a multivariate, engine-specific temporal regression problem; the deployment objective is unseen-engine prediction.	Dataset table and target timeline
Why leakage matters	Random rows are not independent because adjacent cycles from the same engine are highly related.	Engine-level split diagram and engine-overlap test
Data engineering decision	The pipeline learns every transform only from the current fold's training engines.	Data-flow chart plus artifact provenance
Target decision	Training can use capped RUL, but model ranking is based on raw true RUL.	rul_true vs rul_train equations
Evaluation decision	No single score tells the full safety story: use accuracy, asymmetry, end-of-life error, uncertainty, and cost.	Metric table and NASA asymmetry explanation
Reproducibility decision	All teams consume one folds.json and write a common OOF prediction contract.	Manifest/hash fields and output schema

Final readiness checklist

- All data-quality, target, causal-feature, and leakage tests are green.
- Each candidate model uses the identical folds.json manifest hash.
- Every reported cross-validation metric is aggregated from OOF predictions, not fitted predictions.
- Main tables score rul_true; training target use is visible and documented.
- Last-20 uses rul_true ≤ 20; it is not a positional slice.
- NASA error sign is verified by numeric oracle tests.
- Official test has not been queried during development and will be executed once after protocol lock.

Status: this is an engineering and evaluation deliverable. It defines a rigorous protocol but contains no trained-model results, no accuracy claim, and no declared winner.